Graph: DFS, BFS, Connectivity, flood fill

Wattana Jindaluang

Graph traversal

- The most basic graph algorithm that visits nodes of a graph in certain order
- Used as a subroutine in many other algorithms
- We will cover two algorithms
  - Depth-First Search (DFS): uses recursion, stack
  - Breadth-First Search (BFS): uses queue

Depth-First Search

- Traversal can start from any vertex, say v_i.
- v_i is visited and then all vertices adjacent to v_i are traversed recursively using DFS.
- Since, a graph can have cycles -> we must avoid revisiting a node.
- To do this, when we visit a vertex v, we mark it visited.
- A node that has already been marked as visited should not be selected for traversal.
- Marking of visited vertices can be done with the help of a global array visited[ ].
- Array visited[ ] is initialized to false (0).

Depth-First Search (Cont.)

- DFS(v): visits all the nodes reachable from v in depth-first order
  - Mark v as visited
  - For each edge v -> u: // (Explore all the neighbors of v)
    - If u is not visited, call DFS(u)
  - Use non-recursive version if recursion depth is too big
    - Replace recursive calls with a stack

Depth-First Search (Cont.)

```
procedure DFS(w)
        for each vertex x adjacent to w do
                if colour[x] is white then
                        DFS(x)
                end if
        end for
        colour[w] <- black
```

Depth-First Search (Cont.)

```
procedure DFS(w)
    initialize stack S
    push w onto S
    while S not empty do
        x <- pop off S
        if colour[x] = white then
            colour[x] <- black
            for each vertex y adjacent to x do
                if colour[y] is white then
                    push y onto S
```

Breadth-First Search

- BFS(v): visits all the nodes reachable from v in breadth-first order
  - Initialize a queue Q
  - Mark v as visited and push it to Q
  - While Q is not empty:
    - Take the front element of Q and call it w
    - For each edge w -> u: // (Explore all the neighbors of w)
      - If u is not visited, mark it as visited and push it to Q

Breadth-First Search (Cont.)

```
procedure BFS(v)
    Push v on to the tail of Q and set colour[v] <-grey
    while Q is not empty
        Pop vertex w from the head of Q
        for each vertex x adjacent to w do
            if colour[x] is white then
                colour[x] <- grey
                Push x on to the tail of Q
        colour[w] <- black
```

Connectivity

- A graph is said to be **connected** if there is a path between every pair of vertices.
- To check connectivity of a graph, we will try to traverse all nodes using any traversal algorithm.
- After completing the traversal, if there is any node, which is not visited, then the graph is not connected.

Connectivity (Cont.)

- To check this using BFS traversal,
  - Start BFS traversal from any node (say node 0).
  - Track visited vertices using a visited[] array.
  - After BFS, if all vertices are visited, the graph is connected. And, if any vertex remains unvisited, the graph is disconnected.

Connectivity (Cont.)

- Algorithm: Checking Connectivity of a Graph
  - **Step 1:** Start
  - **Step 2:** Input number of vertices V and number of edges E
  - **Step 3:** Create an adjacency list for the graph
  - **Step 4:** For each edge, input the pair of vertices (u, v) and add them to each other's adjacency list
  - **Step 5:** Initialize a visited array of size V with all values set to false
  - **Step 6:** Initialize a queue for BFS traversal
  - **Step 7:** Mark the starting node (vertex 0) as visited
  - **Step 8:** Push the starting node into the queue

Connectivity (Cont.)

- Algorithm: Checking Connectivity of a Graph (Cont.)
  - **Step 9:** While the queue is not empty, repeat steps 10 to 12
  - **Step 10:** Pop a vertex from the queue
  - **Step 11:** For each adjacent vertex of the popped vertex, check if it is unvisited
  - **Step 12:** If unvisited, mark it as visited and enqueue it
  - **Step 13:** After BFS, check if all vertices are visited
  - **Step 14:** If all are visited, the graph is connected; otherwise, it's disconnected
  - **Step 15:** End

flood fill

- Given a 2D grid representing an image, where each element is an integer that denotes the color of a pixel.
- Also there is a coordinates representing the starting pixel (row and column) and an integer, which represents the new color to apply.
- We need to perform a flood fill on the image starting from specified row and column.

flood fill (Cont.)

- It means we must change the color of the starting pixel and all other pixels that are connected to it (directly or indirectly) and have the same original color as the starting pixel.
- Two pixels are considered connected if they are adjacent horizontally or vertically (not diagonally).

flood fill (Cont.)

	0	1	2	3
0	1	1	1	0
1	0	1	1	1
2	1	0	1	1

start at row 1 and column 2

flood fill (Cont.)

	0	1	2	3
0	2	2	2	0
1	0	2	2	2
2	1	0	2	2

output

flood fill (Cont.)

- The idea is to use Depth-First Search (DFS) because DFS explores all connected cells deeply before backtracking, we need to change the color of the starting pixel and all other pixels that are connected to it having the same original color.

flood fill (Cont.)

- So, we can think of it like this:
  - We start from the given pixel (row, column) and store its color as oldColor. Then, we begin a DFS from that cell. For each cell, we check all its four neighboring cells, If the neighbor is inside the grid and has the same color as oldColor, then that means this pixel should also be filled with the new color. We update it with newColor and recursively call DFS for that neighbor to continue spreading the new color further. If a neighbor has a different color or is outside the grid, we simply return without going deeper. Once all connected cells are visited, we'll have our final filled image.

flood fill (Cont.)

- The idea is the same as DFS, but instead of using recursion, we use Breadth-First Search (BFS) because it avoids recursion overhead.
- In BFS, we use a queue and traverse level by level. We start from the given pixel, change its color to the new one, and push it into the queue. Then, for each pixel we pop, we check all four adjacent cells—if any neighbor has the same old color, we update it with the new color and add it to the queue. This continues until all connected pixels are updated.

Copied from

- Jaehyun Park, “Basic Graph Algorithms”, Standford University, 2015.
- Tree and Graph Traversals, Retrieved March 2017, Available at

http://teaching.csse.uwa.edu.au/units/CITS2200/Lectures/Topic12-Traversals.pdf, Accessed April, 2026.

- Kevin Wayne, “Algorithm Design: Graphs”, PEARSON-Addison Wesley, 2005.
- Geeksforgeeks, “Flood Fill Algorithm”, Available at: https://www.geeksforgeeks.org/dsa/flood-fill-algorithm/, Accessed April, 2026.
- tutorialspoint, “C++ Program to Check the Connectivity of Undirected Graph Using BFS”, Available at: https://www.tutorialspoint.com/article/cplusplus-program-to-check-the-connectivity-of-undirected-graph-using-bfs, Accessed April, 2026